Portal Insight Engine Audit

Date: 2026-05-23

Scope: current Insight / AI Coach / Operator Diagnosis / Historical Baseline implementation. This is an audit of existing behavior, not a redesign.

1. Executive Summary

The current Insight system is **partially trustworthy**.

It is trustworthy as a directional, read-only baseline engine when it has complete fills and usable order enrichment. The code has real data-quality warnings, separates the saved Historical Baseline source from native Journal storage, reconstructs positions deterministically, and labels imported entry style as proxy-only.

It is not yet trustworthy enough for hard behavioral diagnosis or trader-grade claims because several displayed metrics depend on fragile assumptions: Hyperliquid fill/order history caps, incomplete `historicalOrders` enrichment, order matching by order id only, boundary position exclusion, no fill de-duplication, UTC time buckets, and uncertain fee semantics. The largest product risk is that the current evidence mix can feed imported baseline trades into Operator Diagnosis when external context is enabled, which blurs the intended boundary between native Journal evidence and historical context.

2. Current Feature Map

UI surfaces:

Surface	Files	Notes
Footer tab system	`src/components/layout/app-shell.tsx`, `src/features/terminal/lib/footer-mode.ts`	AI Coach and Diagnosis are mode-based footer surfaces.
AI Coach / Insight tab	`src/components/layout/app-shell.tsx`, `src/features/ai-coach/lib/observational-coach.ts`, `src/features/ai-coach/lib/ai-warning-aggregation.ts`, `src/features/ai-coach/lib/evidence-source-mix.ts`	Shows active warnings, Context Sources, baseline cards, Journal cards, and evidence-source toggles.
Historical Baseline wizard	`src/components/layout/app-shell.tsx`, `src/features/baseline-import/*`	`Run Baseline Scan`, preview, save, refresh, source card, and fullscreen Baseline Profile.
Baseline source cards	`src/components/layout/app-shell.tsx`, `src/features/baseline-import/storage.ts`	Saved as `portal.baseline-import` in localStorage; one bundle per exchange/environment/account id.
Baseline Profile modal	`src/components/layout/app-shell.tsx`	Shows performance, structure, behavior, asset breakdown, source health, warnings.
Operator Diagnosis tab	`src/components/layout/app-shell.tsx`, `src/features/diagnosis/lib/operator-diagnosis.ts`, `src/features/diagnosis/lib/discipline-plan-activation.ts`	Deterministic report plus review-first Discipline Plan.
Realtime diagnosis warnings	`src/features/diagnosis/lib/diagnosis-realtime-warnings.ts`, `src/features/ai-coach/lib/observational-coach.ts`	Injected into AI Coach warning stream.
Native Journal insights	`src/features/journal/lib/journal-insights.ts`, `src/features/journal/lib/journal-memory-insights.ts`, `src/components/layout/app-shell.tsx`	Hunted/chased, setup tags, memory insights in recap surfaces.
Avatar/Soul summary consumers	`src/features/ai/components/avatar-soul-panel.tsx`, `src/features/avatar/components/avatar-exp-panel.tsx`, `src/components/layout/app-shell.tsx`	Consumes AI/Diagnosis/source summaries, not the core scan engine.
Local debug output	`.portal/audits/insight-baseline-latest.json`, `src/features/baseline-import/debug-output.ts`, `scripts/write-insight-baseline-audit.ts`	New local-only audit summary writer.

3. Data Pipeline

```
Wallet address + environment + date range
-> app-shell baseline wizard builds BaselineImportRequest
```

```
-> runHistoricalBaselineScan()
-> Hyperliquid /info fetches userFillsByTime, historicalOrders, userFunding,
userNonFundingLedgerUpdates, clearinghouseState
-> normalizeUserFills / normalizeHistoricalOrders / normalizeUserFunding / normalizeLedgerUpdates
-> reconstructImportedTrades()
-> buildSessions()
-> attachFunding()
-> buildBaselineProfile()
-> StoredBaselineImportBundle saved to localStorage
-> AI Coach receives baselineProfile when External Context is enabled
-> Operator Diagnosis may receive mapped imported trades when External Context is enabled
-> app-shell renders source cards, Baseline Profile modal, AI Coach cards, warnings, and
Diagnosis report
```

4. Hyperliquid Data Sources

Portal calls the Hyperliquid `info` endpoint directly from the app/runtime code:

Source	Code	Endpoint / payload	Date range	Failure behavior
Fills	`fetchUserFillsByTime` in `src/features/baseline-import/hyperliquid-import-client.ts`	`POST https://api.hyperliquid.xyz/info` or testnet with `{ type: "userFillsByTime", user, startTime, endTime, aggregateByTime: true }`	Uses selected start/end.	Fatal if request fails.
Historical orders	`fetchHistoricalOrders`	`{ type: "historicalOrders", user }`	No start/end sent.	Caught and replaced with `[]`.
Funding	`fetchUserFunding`	`{ type: "userFunding", user, startTime, endTime }`	Uses selected start/end.	Caught and replaced with `[]`.
Ledger	`fetchUserNonFundingLedgerUpdates`	`{ type: "userNonFundingLedgerUpdates", user, startTime, endTime }`	Uses selected start/end.	Caught and replaced with `[]`.
Account snapshot	`fetchClearinghouseState`	`{ type: "clearinghouseState", user }`	Snapshot only.	Caught and replaced with `null`.

Known external limitations:

- Hyperliquid documents `historicalOrders` as returning at most 2,000 recent historical orders.
- Hyperliquid docs list `userFillsByTime`, `historicalOrders`, `userFunding`, and related endpoints as response-size-weighted info endpoints.
- The UI says Hyperliquid exposes the 10,000 most recent fills, while the code warns at 2,000 returned fills. That wording/code mismatch should be resolved against current Hyperliquid docs before redesign.

Sources checked: [Hyperliquid Info endpoint](https://hyperliquid.gitbook.io/hyperliquid-docs/for-developers/api/info-endpoint), [Hyperliquid rate limits and user limits](https://hyperliquid.gitbook.io/hyperliquid-docs/for-developers/api/rate-limits-and-user-limits).

5. Metric Inventory

Trades analyzed

File/Function: `buildBaselineProfile`

Input Data: Reconstructed trades

Logic: `totals.totalTrades = trades.length`

Confidence: Medium

Weak Spots: Includes open trades and reversed fragments; excludes boundary closes.

UI Location: Baseline Profile header, source cards.

Closed trades

File/Function: `buildBaselineProfile`

Input Data: Reconstructed trades

Logic: `status !== "open"`

Confidence: Medium

Weak Spots: `reversed-fragment` counts as closed.

UI Location: Source card secondary copy.

Win rate

File/Function: `buildBaselineProfile`

Input Data: Closed trades

Logic: `wins / closedTrades` using `netPnl > 0`

Confidence: Medium

Weak Spots: Net PnL may not be fee-adjusted correctly; partial history skews rate.

UI Location: Baseline Profile header, AI Coach Historical Performance.

Net after fees

File/Function: `buildBaselineProfile`

Input Data: Closed trades

Logic: Sum `trade.netPnl`; `netPnl = executionClosedPnl + fundingPnl`

Confidence: Low-Medium

Weak Spots: Code labels this after fees but does not subtract `grossFees`; depends on Hyperliquid `closedPnl` semantics.

UI Location: Baseline Profile Performance.

Estimated fees on closed trades

File/Function: `buildBaselineProfile`

Input Data: Closed trades

Logic: Sum `grossFees` across closed trades

Confidence: Medium

Weak Spots: Fee records are fill-level and may be split during reversals; fee currency assumed displayable as USD.

UI Location: Baseline Profile Performance.

Gross before fees

File/Function: `buildBaselineProfile`

Input Data: Closed trades

Logic: `netPnl + closedGrossFees`

Confidence: Low

Weak Spots: Correct only if `netPnl` is already after fees.

UI Location: Baseline Profile Performance.

Average closed trade

File/Function: `buildBaselineProfile`

Input Data: Closed trades

Logic: Average `trade.netPnl`

Confidence: Medium

Weak Spots: Same PnL/fee and incomplete-history risks.

UI Location: Baseline Profile Performance.

Total imported fees for audit

File/Function: `buildBaselineProfile`

Input Data: All trades

Logic: Sum `grossFees` including open trades

Confidence: Medium

Weak Spots: Useful audit count, not a closed-performance metric.

UI Location: Source Health footer.

Short vs long count

File/Function: `buildBaselineProfile` / `groupMetricRows`

Input Data: Closed trades

Logic: Group by `trade.direction`

Confidence: Medium

Weak Spots: Direction is inferred from position sign; boundary/reversal errors can skew.

UI Location: Baseline Profile Structure, AI Coach Long vs Short Bias.

Market vs limit vs mixed

File/Function: `reconstructImportedTrades` / `deriveExecutionMode`

Input Data: Matched entry orders

Logic: Non-reduce, non-TP/SL order types; market+limit => mixed

Confidence: Low-Medium

Weak Spots: Missing orders become unknown; order type parsing is string-based; `historicalOrders` capped.

UI Location: Baseline Profile Structure, AI Coach Entry Proxy.

Likely chased proxy

File/Function: `deriveEntryStyleProxy`

Input Data: Execution mode

Logic: `market -> likely-chased`

Confidence: Low

Weak Spots: Proxy only; market orders are not always emotional chasing.

UI Location: Baseline Profile Structure, AI Coach Entry Proxy.

Likely hunted proxy

File/Function: `deriveEntryStyleProxy`

Input Data: Execution mode

Logic: `limit -> likely-hunted`

Confidence: Low

Weak Spots: Proxy only; limit orders can chase too.

UI Location: Baseline Profile Structure, AI Coach Entry Proxy.

Best time window

File/Function: `buildBaselineProfile`, `getBestReliableBaselineMetricRow`

Input Data: Closed trades grouped by UTC hour bucket

Logic: Best reliable `byTimeOfDay.averageNetPnl`

Confidence: Medium

Weak Spots: Uses UTC buckets named like sessions; modal requires 10 trades but AI Coach card does not.

UI Location: Baseline Profile Behavior, AI Coach Timing.

Worst time window

File/Function: `getWorstReliableBaselineMetricRow`

Input Data: Closed trades grouped by UTC hour bucket

Logic: Worst reliable `byTimeOfDay.averageNetPnl`

Confidence: Medium

Weak Spots: Same UTC/reliability mismatch.

UI Location: Baseline Profile Behavior.

Best hold range

File/Function: `buildBaselineProfile`, `getBestReliableBaselineMetricRow`

Input Data: Closed trades with duration

Logic: Best reliable hold bucket by avg PnL

Confidence: Medium

Weak Spots: Duration depends on reconstructed entry/exit; open trades excluded.

UI Location: Baseline Profile Behavior, AI Coach Timing.

Worst hold range

File/Function: `getWorstReliableBaselineMetricRow`

Input Data: Closed trades with duration

Logic: Worst reliable hold bucket by avg PnL

Confidence: Medium

Weak Spots: Same duration/reliability risks.

UI Location: Baseline Profile Behavior.

Size-after-loss events

File/Function: `buildSizeAfterLoss`

Input Data: Chronological closed trades

Logic: Previous trade negative and next max size >= 10% larger

Confidence: Medium

Weak Spots: Uses max base size, not notional/risk/leverage; cross-asset comparisons are weak.

UI Location: Baseline Profile Behavior, AI Coach Behavior Signals.

High-density sessions

File/Function: `buildSessions`, `buildBaselineProfile.sessionBehavior`

Input Data: Reconstructed closed trades

Logic: Session gap > 90m starts new session; sessions with >5 trades

Confidence: Medium

Weak Spots: Uses closed trades and reconstructed session ids; overlap/open positions can distort.

UI Location: Baseline Profile Behavior, AI Coach Behavior Signals.

Short-gap trades

File/Function: `buildBaselineProfile.sessionBehavior`

Input Data: Closed trades sorted by entry

Logic: Entry <= 15m after previous end

Confidence: Medium

Weak Spots: Uses reconstructed end time; not symbol-aware.

UI Location: AI Coach Behavior Signals / debug output.

Asset breakdown

File/Function: `buildBaselineProfile.byAsset`

Input Data: Closed trades

Logic: Group by `coin`, avg PnL and counts

Confidence: Medium

Weak Spots: Coin strings can include unusual asset ids; incomplete history skews.

UI Location: Baseline Profile Asset Breakdown, AI Coach Assets.

Aggregated fills

File/Function: `runHistoricalBaselineScanFromRecords`

Input Data: Normalized fills

Logic: `sourceHealth.aggregatedFillCount = userFills.length`

Confidence: High

Weak Spots: Does not prove completeness; Hyperliquid caps still apply.

UI Location: Source Health.

Recommendation trades

File/Function: `runHistoricalBaselineScanFromRecords`

Input Data: Reconstructed trades

Logic: `sourceHealth.recommendationTradeCount = trades.length`

Confidence: Medium

Weak Spots: Includes open/reversed fragments; name implies stronger quality than warranted.

UI Location: Source Health.

Boundary fills excluded

File/Function: `reconstructImportedTrades`

Input Data: Fills where `currentTrade === null && startPosition != 0`

Logic: Count and skip fills from positions opened before window

Confidence: Medium

Weak Spots: Correctly avoids phantom trades but can hide real PnL inside selected date range.

UI Location: Source Health, warnings.

Order matching matched/unmatched

File/Function: app-shell derived counts

Input Data: Reconstructed trades

Logic: Matched = sum `rawOrderMatches.length`; unmatched = trades with zero matches

Confidence: Medium

Weak Spots: Matched count is order-match rows, not trade count; missing individual order ids can still exist when trade has some matches.

UI Location: Source Health.

Manual entry risk warning

File/Function: `deriveDiagnosisRealtimeWarningsV1`, `buildWarningFromDisciplineBlock`

Input Data: Operator diagnosis finding, current action, discipline logs

Logic: Warns on manual market action when manual entries underperform or discipline confirmation required

Confidence: Medium

Weak Spots: Depends on native/imported evidence mix and entrySource mapping.

UI Location: AI Coach Active Warnings.

Historical performance card

File/Function: `buildBaselinePerformanceCard`

Input Data: Baseline profile totals

Logic: Shows total trades, win rate, net, closed fees

Confidence: Medium

Weak Spots: Does not surface data-quality warnings inside the card; source card/modal carry warnings.

UI Location: AI Coach cards.

Historical performance warning

File/Function: app-shell source card / modal data-quality copy

Input Data: Baseline `dataQuality`

Logic: Source card turns warning tone when dataQuality exists; modal says directional history

Confidence: Medium

Weak Spots: AI Coach baseline cards can still look confident while source warnings exist.

UI Location: Context Sources and Baseline Profile.

Native hunted/chased comparison

File/Function: `deriveJournalInsightsV1`

Input Data: Native Journal closed entries

Logic: Requires minimum 3 per style, compares average realized PnL

Confidence: Medium-High

Weak Spots: Native labels depend on user/app capture quality.

UI Location: Journal recap Insights, AI Coach Entry Style card.

Setup tags

File/Function: `deriveJournalInsightsV1`

Input Data: Native Journal setup tags

Logic: Requires minimum 3 tagged trades, ranks by average PnL, suppresses ties

Confidence: Medium

Weak Spots: No imported setup tags; user-entered tags can be inconsistent.

UI Location: Journal recap, AI Coach Setup Tags.

6. Trade Reconstruction Audit

Reconstruction is deterministic and fill-led:

1. Fills are sorted by coin, time, order id, trade id.
2. Each fill becomes signed size from side: buy positive, sell negative.
3. Position before is `fill.startPosition`, or the active reconstructed position, or zero.
4. Position after is `before + signedSize`, rounded to 12 decimals.

5. If there is no active trade but `before` is non-zero, the fill is counted as a boundary fill and skipped unless it reverses into a new in-window position.
6. Adds increase same-sign exposure; reduces decrease same-sign exposure; zero closes the trade; sign flips split the fill into close and new open fragments.
7. Entry/exit prices are volume-weighted by fill price and size.
8. Order matches are attached after reconstruction by `orderId`.
9. Funding records are summed inside `[entryTime, endTime]` for the same coin.
10. Sessions are grouped with a 90-minute inactivity gap.

Where this can break:

- Position opened before selected range: boundary fills are excluded, so realized PnL inside the window may be missing from recommendations.
- Partial fills and scale-outs: handled mechanically, but the system does not distinguish intent, risk, or separate planned scale actions.
- Overlapping same-coin positions: collapsed into one active trade per coin, which matches a net-position model but loses sub-position intent.
- Cross-asset sizing: size-after-loss compares raw base size, not notional or risk.
- Missing `startPosition`: falls back to active reconstructed position and can drift if previous fills are missing.
- Duplicate fills: no de-duplication pass exists.
- Invalid timestamps: normalization can fall back to `Date.now()`, which can silently corrupt buckets.
- Order matching: order map is keyed only by numeric order id; there is no extra coin/time/status validation.
- Fees: reversal fill splitting prorates fee by size; this is reasonable but not exchange-authoritative.

7. Behavior Classifier Audit

Classifier	Logic	Confidence	Notes
Chased	Entry execution mode `market` maps to `likely-chased`.	Low	Correctly labeled proxy, not truth.
Hunted	Entry execution mode `limit` maps to `likely-hunted`.	Low	Correctly labeled proxy, not truth.
Manual entry risk	Operator Diagnosis flags manual entries if `entrySource` is market/manual, sample >=2, net negative, and underperforms all trades; realtime warning fires on manual market current action.	Medium	Stronger with native Journal than imported baseline because imported entry source is derived from execution mode.
Size after loss	Next closed trade max size is >=10% larger after a losing closed trade.	Medium	Uses raw size, not notional/risk.
High-density session	More than 5 closed trades in a 90-minute-session grouping.	Medium	Uses reconstructed closed trades only.
Time window	Baseline uses UTC buckets: overnight, premarket, cash, afternoon, evening. Diagnosis uses local `getHours()` one-hour buckets.	Low-Medium	Mixed UTC/local assumptions are a redesign blocker.
Hold range	Duration buckets: `<5m`, `5-15m`, `15-60m`, `1-4h`, `4-24h`, `24h+`, `open`.	Medium	Depends on reconstructed close times.
Market/limit/mixed	Matched non-reduce entry orders are parsed from orderType strings.	Low-Medium	Missing orders are common at caps; string parsing can misclassify.
Long/short bias	Group closed reconstructed trades by direction and compare avg PnL.	Medium	Direction is net-position-derived.
Asset performance	Group closed reconstructed trades by `coin`.	Medium	Needs complete history and normalized asset ids.
Native setup tags	Native Journal-only, user/app-entered tags ranked after minimum sample.	Medium	Imported baseline has no setup tags.

8. Warning / Diagnosis Audit

AI Coach warnings come from three places:

- Session log risk/discipline events in `observational-coach.ts`.
- Trade behavior checks such as sizing up after a loss.
- Operator Diagnosis realtime warnings from `diagnosis-realtime-warnings.ts`.

Ranking/deduping:

- Warnings are keyed by trigger/rule/symbol.
- Latest warning for the same id wins.
- Dismissed ids are filtered out.
- TTL warnings expire after their `ttlMs`.
- Sort order is `critical`, then `warning`, then `info`, then newest timestamp.

Operator Diagnosis generation:

- `deriveOperatorDiagnosisV1` converts Journal entries into closed trade summaries.
- By default, the helper excludes imported entries.
- In app-shell, `includeImportedEntries` is set from `aiEvidenceSourceMix.externalContext`, and `mapBaselineImportTradesForDiagnosis()` maps baseline trades into diagnosis-shaped entries.
- Findings include post-loss behavior, repeated Override Bonds, ignored/dismissed AI warnings, broken plans, weak time windows, manual entry weakness, overtrading days, followed-plan positives, and Magic Trade positives.
- Confidence is sample-size only: low under 5, medium 5-19, high 20+.

Key issue: sample-size confidence does not account for source quality. A 30-trade imported baseline with order caps and boundary exclusions can produce `high` diagnosis confidence if external context is enabled. Future confidence must combine sample size with data quality.

9. Known Data Quality Problems

- `historicalOrders` is fetched without date parameters and may be capped at 2,000 recent orders.
- `historicalOrders`, funding, ledger, and account snapshot failures are silently degraded to empty/null sources.
- `missing-order-matches` warning can coexist with zero unmatched trades because unmatched trade count only counts trades with no matches at all.
- Boundary fills are excluded from recommendation-grade trades, which protects against phantom positions but omits in-window closes of pre-window positions.
- Open trades inside the window are counted in total trades but excluded from closed performance metrics.
- Positions opened before the selected date window are not reconstructed fully.
- Partial fills, scale-ins, and scale-outs are handled as net position math, not strategy intent.
- Overlapping same-coin positions are collapsed into one net trade.
- Duplicate fills are not deduped.
- Invalid or missing timestamps can become `Date.now()`.
- Fee accounting is potentially mislabeled: current `net after fees` does not subtract `grossFees` unless Hyperliquid `closedPnl` is already fee-net.
- Funding is attached by coin and trade time window; overlapping same-coin trades could double-count or misattribute funding.
- Market/limit classification depends on successful order enrichment and string parsing.

- Wallet casing is accepted, but storage id lowercases the account; display and request preserve trimmed input casing.
- Baseline date input is parsed as UTC day boundaries, while Diagnosis time-window buckets use local browser/server `getHours()`.
- UI copy says 10,000 most recent fills; code data-quality warning fires at 2,000 fills.
- AI Coach baseline cards do not carry source-warning state strongly enough; the source orb/modal do.
- Imported baseline can feed Operator Diagnosis through the evidence-source mix, which risks presenting external proxy data as diagnosis evidence.

10. Recommended Fixes

Must fix before redesign:

- Confirm Hyperliquid `closedPnl` fee semantics and correct `net after fees` / `gross before fees` labels or formulas.
- Add source-quality-adjusted confidence so capped/order-incomplete baselines cannot become high-confidence diagnoses.
- Keep imported Historical Baseline out of Operator Diagnosis by default, or label external-context diagnosis separately from native Operator Diagnosis.
- Reconcile Hyperliquid fill/order cap copy and warnings with current docs.
- Add de-duplication and timestamp validation in normalization; do not fall back to `Date.now()` for historical records.

Should fix soon:

- Make matched/unmatched order metrics count linked order ids, fully matched trades, partially matched trades, and no-match trades separately.
- Surface data-quality warnings directly on baseline-powered AI Coach cards.
- Add notional/risk-aware size-after-loss classification.
- Split UTC exchange-time buckets from trader-local time buckets and display which one is used.
- Add tests for duplicate fills, invalid timestamps, partially matched multi-order trades, overlapping same-coin positions, and fee math.

Nice to have later:

- Add paginated/windowed Hyperliquid fill walking with explicit completeness status.
- Persist scan provenance and source-health snapshots in a durable backend.
- Add confidence per metric, not only per diagnosis.
- Add asset normalization for unusual Hyperliquid coin labels.
- Add a user-facing "why this insight exists" evidence drawer.

11. Proposed Confidence Model

High confidence:

- Closed trade sample is large enough for the specific metric.
- Fill scan did not hit caps.
- Order enrichment is complete or not needed for that metric.
- No boundary fills affect the metric.
- Fee semantics are confirmed.
- Metric is based on native Journal evidence or explicitly complete imported history.

Medium confidence:

- Sample is adequate but some warnings exist.

- Metric does not rely heavily on order classification.
- Boundary/open-position effects are limited and disclosed.

Low confidence:

- Order matches are incomplete.
- Metric relies on market/limit proxy classification.
- Date range may be truncated.
- Time-zone assumptions affect the result.
- Sample is small or bucket is below 10 trades.

Insufficient evidence:

- No closed trades.
- No reliable source data.
- Missing timestamps/prices/sizes needed for the metric.
- Data-quality warnings directly invalidate the metric.

12. Files Reviewed

- `AGENTS.md`
- `docs/audits/portal-insight-legitimacy-validation-2026-05-22.md`
- `docs/avatar-system/avatar-system-audit.md`
- `docs/launch/v1-known-limitations.md`
- `docs/launch/v1-screenshot-checklist.md`
- `src/components/layout/app-shell.tsx`
- `src/features/ai-coach/lib/ai-warning-aggregation.ts`
- `src/features/ai-coach/lib/ai-warning-aggregation.test.ts`
- `src/features/ai-coach/lib/evidence-source-mix.ts`
- `src/features/ai-coach/lib/evidence-source-mix.test.ts`
- `src/features/ai-coach/lib/observational-coach.ts`
- `src/features/ai-coach/lib/observational-coach.test.ts`
- `src/features/ai/components/avatar-soul-panel.tsx`
- `src/features/avatar/components/avatar-exp-panel.tsx`
- `src/features/baseline-import/baseline-import.test.ts`
- `src/features/baseline-import/build-baseline-profile.ts`
- `src/features/baseline-import/debug-output.ts`
- `src/features/baseline-import/hyperliquid-import-client.ts`
- `src/features/baseline-import/index.ts`
- `src/features/baseline-import/normalize-import.ts`
- `src/features/baseline-import/reconstruct-trades.ts`
- `src/features/baseline-import/storage.ts`
- `src/features/baseline-import/types.ts`

- src/features/diagnosis/lib/diagnosis-realtime-warnings.ts
- src/features/diagnosis/lib/diagnosis-realtime-warnings.test.ts
- src/features/diagnosis/lib/discipline-plan-activation.ts
- src/features/diagnosis/lib/discipline-plan-activation.test.ts
- src/features/diagnosis/lib/operator-diagnosis.ts
- src/features/diagnosis/lib/operator-diagnosis.test.ts
- src/features/journal/lib/journal-insights.ts
- src/features/journal/lib/journal-insights.test.ts
- src/features/journal/lib/journal-memory-insights.ts
- src/features/journal/lib/journal-memory-insights.test.ts
- package.json
- scripts/run-src-tests.mjs
- scripts/write-insight-baseline-audit.ts

13. Local Debug Output

Added a local-only writer:

```
npm run audit:insight-baseline -- --address 0x... --environment mainnet --start 2026-04-23 --end
2026-05-22
```

It writes:

```
.portal/audits/insight-baseline-latest.json
```

The generated file includes source counts, fill/order/funding/ledger counts, matched/unmatched reconstruction counts, reconstructed trade counts, excluded boundary fills, warnings, classifier outputs, reconstruction confidence counts, and headline totals. It does not include private keys or secrets.

The sample generated during this audit used the public validation wallet already present in `docs/audits/portal-insight-legitimacy-validation-2026-05-22.md` and produced:

- 57 fills
- 2,000 historical orders
- 206 funding records
- 9 ledger updates
- 28 reconstructed trades
- 1 boundary fill excluded
- 56 matched order rows
- 0 fully unmatched trades
- 3 warnings: missing order matches, boundary fills excluded, order cap hit